LoopKaka

ULTIMATE RAG COURSE

From Basics to Production — Build RAG the Right Way

loopkaka

CHAPTERS



Build Baseline RAG 01

Better Ingestion and Chunking 02

Advanced Retrieval 03

Query and Context Engineering 04

RAG Evaluation and Optimization 05

LangGraph and RAG 06

Production RAG 07

CHAPTER 1 *(8 Episodes)*



Understand the fundamentals of Retrieval-Augmented Generation (RAG) and build a complete baseline RAG application from scratch.

- **Document Loading** — Load documents into the application.
- **Document Chunking** — Split documents into smaller, manageable chunks.
- **Embeddings** — Convert text into vector representations.
- **Vector Database** — Store and manage document embeddings using Qdrant.
- **Semantic Retrieval** — Retrieve relevant chunks based on a user's query.
- **Answer Generation** — Use OpenAI LLMs to generate answers from retrieved context.
- **End-to-End RAG** — Connect all components into a working pipeline.

CHAPTER 2 *(3 Episodes)*



Learn how to prepare different types of documents for effective retrieval in a RAG application.

- **Document Loading** — Load different data sources and handle PDF and DOCX loading challenges.
- **Metadata** — Add useful metadata to documents and their chunks.
- **Chunking Strategies** — Explore fixed-size, recursive, sentence-based, paragraph-based, token-based, structure-aware, and semantic chunking.
- **Strategy Selection** — Understand the trade-offs between different chunking approaches.

CHAPTER 3 *(6 Episodes)*



Improve retrieval quality by addressing the limitations of baseline RAG.

- **Retrieval Problems** — Understand common retrieval failures and their impact.
- **Metadata Filtering** — Filter retrieved information using metadata.
- **Top-K & Similarity Threshold** — Control how many chunks are retrieved and how relevant they should be.
- **Hybrid Search** — Combine dense vector search with BM25 sparse retrieval.
- **Reciprocal Rank Fusion (RRF)** — Combine ranked results from multiple retrieval methods.
- **Reranking** — Reorder retrieved chunks to improve relevance.
- **Optimized Retrieval Pipeline** — Integrate the techniques into one pipeline.

CHAPTER 4 *(6 Episodes)*



Improve how user questions are interpreted and how retrieved information is selected for answer generation.

- **Query Rewriting** — Reformulate questions to improve retrieval.
- **Multi-Query Retrieval** — Generate multiple search queries to explore different perspectives.
- **Context Selection** — Select the most useful chunks for answering a question.
- **Context Compression** — Reduce unnecessary context while preserving relevant information.
- **MMR** — Balance relevance and diversity in retrieved results.
- **Grounded Answers & Citations** — Generate answers grounded in retrieved evidence and provide citations.

CHAPTER 5 *(7 Episodes)*



Measure RAG quality and use evaluation results to improve accuracy, cost, and latency.

- **Evaluation Fundamentals** — Understand why RAG systems need evaluation.
- **Evaluation Dataset** — Create a dataset for testing RAG performance.
- **Retrieval Metrics** — Evaluate retrieval using Hit Rate, Recall, and Mean Reciprocal Rank (MRR).
- **Generation Metrics** — Evaluate correctness, faithfulness, and answer relevance.
- **Ragas** — Run practical RAG evaluations.
- **Optimization** — Improve the pipeline based on evaluation results.
- **Cost & Latency** — Measure the cost and response time of RAG requests.

CHAPTER 6 *(3 Episodes)*



Use LangGraph to build a structured RAG workflow with conditional execution and corrective retrieval.

- **Basic Graph RAG** — Build a basic RAG workflow using LangGraph.
- **Conditional Retrieval** — Route execution based on retrieved context.
- **Retrieval Grading** — Evaluate whether retrieved information is relevant to the user's question.
- **Corrective RAG** — Rewrite search queries and retry retrieval when the initial results are insufficient.
- **Fallback Handling** — Return a fallback response when relevant information cannot be found within the retry limit.

CHAPTER 7 *(3 Episodes)*



Prepare your RAG application for integration, monitoring, and deployment.

- **FastAPI RAG API** — Expose the RAG pipeline through REST endpoints.
- **LangSmith Tracing & Monitoring** — Inspect RAG execution, model calls, and errors.
- **Deployment** — Understand how to take the RAG application toward deployment using practical deployment resources.

LEARNING TIPS

How to Get the Most Out of This Course?



- **Practice** along with me.
- **Research** the topics we discuss.
- Prepare your own **notes**.
- **Have questions?** Connect with me on **Discord** or **LinkedIn**, or leave a **comment** on the video. **Link in description.**

TECHNOLOGY STACK



Python 3+	01	Qdrant	05
OpenAI	02	Ragas	06
LangChain & LangGraph	03	Docker	07
LangSmith	04	Visual Studio Code	08

TARGET AUDIENCE

Who Should Watch This Course?



- Python developers interested in GenAI.
- Backend and full-stack developers.
- Developers building AI applications.
- Aspiring GenAI and RAG Engineers.
- Developers interested in RAG evaluation and optimization.
- Basic Python knowledge recommended.

COURSE RESOURCES



- Find the source code and notes on **GitHub**.
- GitHub link in **description**.

EPISODE RELEASE SCHEDULE

- New episodes will be published every 2 days.
- Example: Monday → Wednesday → Friday → Sunday.

LoopKaka

THANK YOU





